pi-artifacts

Claude-style live artifacts, rebuilt on Cloudflare: an agent builds dashboards that fetch real-time MCP data every time they are viewed.

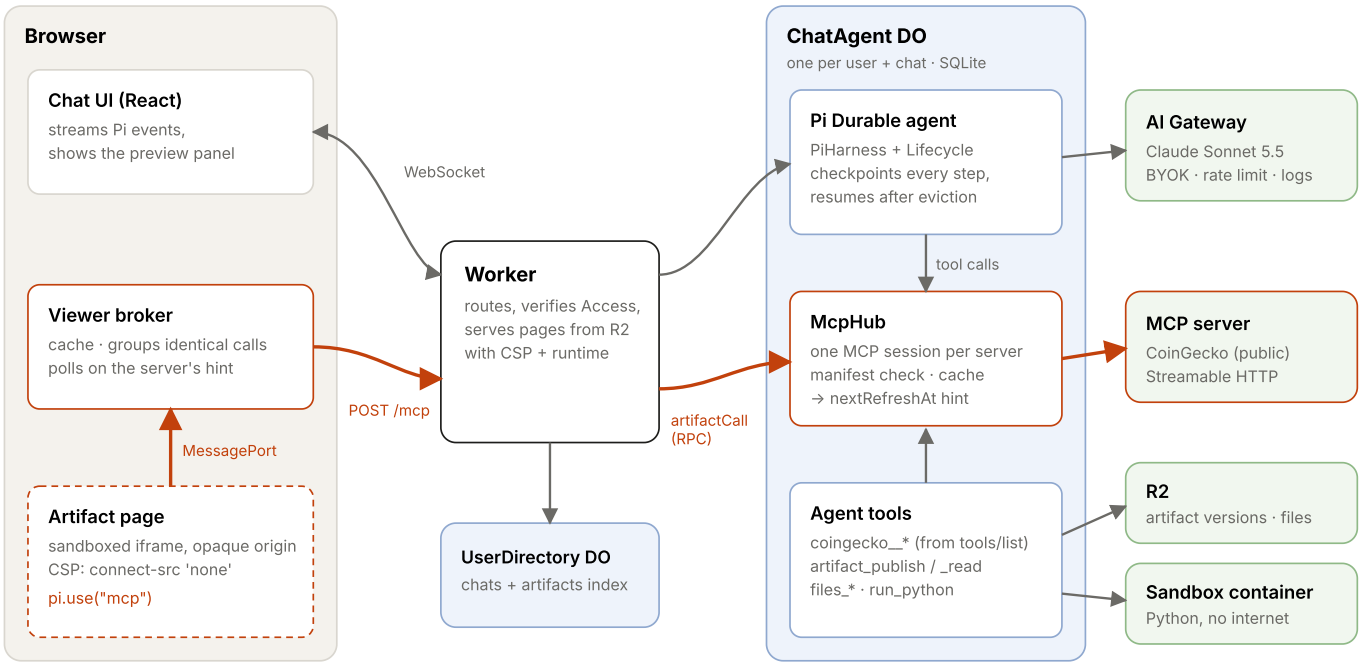
You chat with a Pi agent. It finds the data through MCP tools and writes a self-contained HTML page. It then publishes the page as a versioned *artifact* that appears beside the chat and can be shared by link. Each time someone opens the page, it calls the MCP tools declared at publish time and keeps the data current. The page API has the same shape as Claude's `claude.use("mcp")`. The demo uses CoinGecko's free public MCP server.

0 tokens
per live data call: no model in the path

~1 s
page → MCP → page, measured against CoinGecko

30 s
polling floor, timed by the server's refresh hint

1 DO
per user + chat: the agent, MCP sessions, cache



Orange: the live-data path a published page uses at view time, with no model involved. Grey: the agent building pages.

How a live call travels

1. **Publish.** The agent makes one real read-only MCP call to learn the data's shape. It writes the page and calls `artifact_publish` with a manifest such as `[{server: "CoinGecko", tools: ["execute"]}]`. The manifest is checked against the server's `tools/list`, and the HTML and manifest are stored in R2 as a new version.
2. **View.** The Worker serves the page with the runtime inlined and a CSP that blocks all network access. The viewer frames it in a sandbox with an opaque origin. The page says hello, and the viewer answers its own frame's first hello with a `MessagePort`.
3. **Watch.** The page calls `mcp.watchTool(server, tool, input, onEvent, {refetchInterval})`. The broker shows the cached result at once, then sends `POST /api/artifacts/:id/mcp`.
4. **Check and call.** The Worker passes the call by RPC to the chat's Durable Object. It checks the version's manifest, applies a per-viewer rate limit, and allows tools that aren't read-only only for the author. It then calls the tool through the *same* McpHub session the agent used: the result is served from the cache or fetched, and identical calls in flight share one request.
5. **Refresh.** The response carries `storedAt` and `nextRefreshAt`. The broker polls again just after that time, pauses while the tab is hidden, and delivers each new result to every watch on that call.

Design decisions

DECISION	WHY	TRADE-OFF
The page reaches the chat's own MCP connection	The page sees exactly what the agent saw; no model, no tokens, deterministic.	Calls run as the author, so sharing a page shares what its manifest allows (Claude uses each viewer's connectors).
A manifest for each published version	The page can't widen its own access; tool names are checked against the server at publish.	A new tool means a new version.
Polling timed by a server hint, not WebSocket push	Within about a second of push, with no sockets, timers or reconnect logic. The upstream data is already 18–32 s old.	Revisit for sources that actually stream.
The same API shape as Claude's runtime	Models already write `use("mcp")` / `watchTool` pages; the contract is inlined into the system prompt.	Has to track a contract owned elsewhere.
One Durable Object per chat, holding the Pi agent and the MCP sessions	Single-threaded state, SQLite checkpoints, and a natural place for the cache.	Page traffic shares an object with the agent loop. The cache absorbs it; the hub can move to a per-user object.

Stack

- Workers + Durable Objects, Pi Durable via the Agents SDK `PiHarness`
- AI Gateway → Claude Sonnet 5.5 (BYOK)
- MCP SDK client (Streamable HTTP) → CoinGecko
- R2 for versions and files; Containers (Sandbox SDK) for Python
- React + Vite, deployed with the `cf` CLI

Status and next

- Deployed; an agent built a live dashboard and revised it as v2
- Cloudflare Access sign-in; files encrypted per user
- Next: a screenshot self-check before publishing, MCP servers with OAuth, push for streaming sources
- `run_python` not yet exercised in production